

Name: Sameer Almadani

Student ID: 2230001818

Classification and Evaluation in NLP

Text classification is a common Natural Language Processing (NLP) task where a model assigns a predefined category or label to a piece of text.

Examples include:

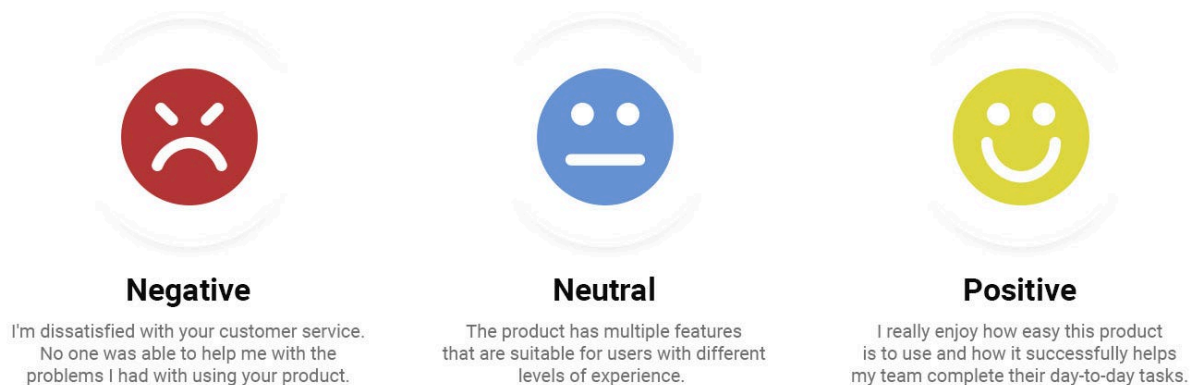
- Spam detection
- Sentiment analysis
- News categorization

In this lab, we will learn how to build and evaluate a text classification model.

Case Study: Sentiment Analysis

Sentiment analysis, also known as opinion mining, its goal is to understand the information present in the text and categorize it as positive, negative, or neutral.

Sentiment Analysis



Disneyland Reviews Dataset

In this lab we'll look into Disneyland Reviews dataset.

*Link: <https://www.kaggle.com/datasets/arushchillar/disneyland-reviews> (<https://www.kaggle.com/datasets/arushchillar/disneyland-reviews>)

The dataset includes 42,000 reviews of 3 Disneyland branches - Paris, California and Hong Kong, posted by visitors on Trip Advisor.

Column Description:

- Review_ID: unique id given to each review
- Rating: ranging from 1 (unsatisfied) to 5 (satisfied)
- Year_Month: when the reviewer visited the theme park
- Reviewer_Location: country of origin of visitor
- Review_Text: comments made by visitor
- Disneyland_Branch: location of Disneyland Park

```
%pip install pandas nltk scikit-learn certifi

# Load the dataset
import pandas as pd

data = pd.read_csv('DisneylandReviews.csv', encoding='latin-1')
data
```

```
Requirement already satisfied: pandas in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (3.0.0)
Requirement already satisfied: nltk in /Users/almadani/Desktop/study/nlp/nlp
lab/.venv/lib/python3.13/site-packages (3.10.3)
Requirement already satisfied: scikit-learn in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (1.8.0)
Requirement already satisfied: certifi in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages
(2026.5.20)
Requirement already satisfied: numpy>=1.26.0 in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from
pandas) (2.4.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
/Users/almadani/Library/Python/3.13/lib/python/site-packages (from pandas)
(2.9.0.post0)
Requirement already satisfied: defusedxml in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from
nltk) (0.7.1)
Requirement already satisfied: click in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from
nltk) (8.4.2)
Requirement already satisfied: joblib in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from
nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in
/Users/almadani/Library/Python/3.13/lib/python/site-packages (from nltk) (2026.7.19)
Requirement already satisfied: tqdm in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from
nltk) (4.67.3)

Requirement already satisfied: scipy>=1.10.0 in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from
scikit-learn) (1.17.1)
Requirement already satisfied: threadpoolctl>=3.2.0 in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from
scikit-learn) (3.6.0)
Requirement already satisfied: six>=1.5 in
/Users/almadani/Library/Python/3.13/lib/python/site-packages (from python-
dateutil>=2.8.2->pandas) (1.17.0)
```

[notice] A new release of pip is available: 25.2 -> 26.2.1

[notice] To update, run: /Users/almadani/Desktop/study/nlp/nlp lab/.venv/bin/python -m
pip install --upgrade pip

Note: you may need to restart the kernel to use updated packages.

	Review_I D	Ratin g	Year_Mon th	Reviewer_Locati on	Review_Text	Branch
0	6707721 42	4	2019-4	Australia	If you've ever been to Disneyland anywhere you...	Disneyland_HongK ong
1	6706827 99	4	2019-5	Philippines	Its been a while since d last time we visit HK...	Disneyland_HongK ong
2	6706232 70	4	2019-4	United Arab Emirates	Thanks God it wasn t too hot or too humid wh...	Disneyland_HongK ong

	Review_ID	Rating	Year_Month	Reviewer_Location	Review_Text	Branch
3	670607911	4	2019-4	Australia	HK Disneyland is a great compact park. Unfortu...	Disneyland_HongKong
4	670607296	4	2019-4	United Kingdom	the location is not in the city, took around 1...	Disneyland_HongKong
...	...	...	...	...	...	...
42651	1765031	5	missing	United Kingdom	i went to disneyland paris in july 03 and thou...	Disneyland_Paris
42652	1659553	5	missing	Canada	2 adults and 1 child of 11 visited Disneyland ...	Disneyland_Paris
42653	1645894	5	missing	South Africa	My eleven year old daughter and myself went to...	Disneyland_Paris
42654	1618637	4	missing	United States	This hotel, part of the Disneyland Paris compl...	Disneyland_Paris
42655	1536786	4	missing	United Kingdom	I went to the Disneyparis resort, in 1996, wit...	Disneyland_Paris

42656 rows × 6 columns

```
#Check for missing values
data.isna().sum()
```

```
Review_ID      0
Rating         0
Year_Month     0
Reviewer_Location 0
Review_Text    0
Branch         0
dtype: int64
```

```
# Drop rows with missing values in the relevant columns
data = data.dropna(subset=['Review_Text', 'Rating'])
```

```
# Map star ratings to sentiments
data['Sentiment'] = data['Rating'].apply(lambda rating: 'positive' if rating > 3 else
('negative' if rating < 3 else 'neutral'))
data.head()
```

	Review_ID	Rating	Year_Month	Reviewer_Location	Review_Text	Branch	Sentiment
0	670772142	4	2019-4	Australia	If you've ever been to Disneyland anywhere you...	Disneyland_HongKong	positive
1	670682799	4	2019-5	Philippines	Its been a while since d last time we visit HK...	Disneyland_HongKong	positive
2	670623270	4	2019-4	United Arab Emirates	Thanks God it wasn't too hot or too humid wh...	Disneyland_HongKong	positive
3	670607911	4	2019-4	Australia	HK Disneyland is a great compact park. Unfortu...	Disneyland_HongKong	positive
4	670607296	4	2019-4	United Kingdom	the location is not in the city, took around 1...	Disneyland_HongKong	positive

```
# Import necessary libraries
import re
import nltk

def preprocess_text(text):

    # 1. Convert text to lowercase
    text = text.lower()

    # 2. Remove punctuation and special characters
    text = re.sub(r'[^a-z\s]', '', text)

    # 3. Tokenize the text
    words = text.split()

    return ' '.join(words)

# Apply preprocessing to the review text
data['Review_Text'] = data['Review_Text'].apply(preprocess_text)

# Display cleaned reviews
data[['Review_Text', 'Sentiment']].head()
```

	Review_Text	Sentiment
0	if youve ever been to disneyland anywhere youl...	positive
1	its been a while since d last time we visit hk...	positive
2	thanks god it wasnt too hot or too humid when...	positive
3	hk disneyland is a great compact park unfortun...	positive
4	the location is not in the city took around ho...	positive

```
# Import necessary libraries
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

# Split the dataset into training and testing sets
train_data, test_data, train_labels, test_labels = train_test_split(data['Review_Text'],
data['Sentiment'], test_size=0.2, random_state=42)

# TF-IDF Vectorization
tfidf_vectorizer = TfidfVectorizer(max_features=1000) # You can adjust max_features based on
your dataset size - limit vocabulary to the 1000 most informative tokens
train_vectors = tfidf_vectorizer.fit_transform(train_data)
test_vectors = tfidf_vectorizer.transform(test_data)
```

```
print(f"train_data size: {train_data.shape}")
print(f"test_data size: {test_data.shape}")
```

```
train_data size: (34124,)
test_data size: (8532,)
```

```

from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# Train the Support Vector Machine (SVM) classifier
svm_classifier = SVC(kernel='linear')
svm_classifier.fit(train_vectors, train_labels)

# Predictions on the test set
predictions = svm_classifier.predict(test_vectors)

# Evaluate the model
accuracy = accuracy_score(test_labels, predictions)
print(f"Accuracy: {accuracy:.2f}")

# Display classification report
print("Classification Report:")
print(classification_report(test_labels, predictions))

```

Accuracy: 0.84

Classification Report:

	precision	recall	f1-score	support
negative	0.64	0.54	0.58	720
neutral	0.48	0.18	0.26	1035
positive	0.88	0.98	0.92	6777
accuracy			0.84	8532
macro avg	0.66	0.56	0.59	8532
weighted avg	0.81	0.84	0.81	8532

Use Case: Amazon reviews of unlocked phone analysis

The objective of this use case is to conduct sentiment analysis on Amazon reviews of unlocked phones, categorizing reviews into three classes: positive, negative, and neutral. The goal is to gain a comprehensive understanding of customer opinions and sentiments regarding various unlocked phone models.

- Dataset link: <https://www.kaggle.com/datasets/PromptCloudHQ/amazon-reviews-unlocked-mobile-phones> (<https://www.kaggle.com/datasets/PromptCloudHQ/amazon-reviews-unlocked-mobile-phones>)

Task1: Load the data and do 5 different preprocessing steps on it

Task2: Use a proper train and test split

Task3: Do Feature Extraction Using TF-IDF

Task4: Train a Naive Bayes Classifier

Task5: Evaluate the model on the Test Set

Task6: Print the Confusion Matrix

```

import pandas as pd
import re
import os
import certifi
import nltk
os.environ.setdefault('SSL_CERT_FILE', certifi.where())
nltk.download('stopwords')
from nltk.corpus import stopwords

reviews = pd.read_csv('Amazon_Unlocked_Mobile.csv')
reviews = reviews.dropna(subset=['Reviews', 'Rating']).copy()
reviews['Sentiment'] = reviews['Rating'].apply(
    lambda rating: 'positive' if rating > 3 else ('negative' if rating < 3 else 'neutral'))
stop_words = set(stopwords.words('english')) - {'no', 'not', 'nor'}

def clean_review(text):
    text = text.lower()
    text = re.sub(r'<[^>]+>|https?://\S+|www\.\S+', ' ', text)
    text = re.sub(r'[^a-z\s]', ' ', text)
    words = text.split()
    words = [word for word in words if word not in stop_words]
    return ' '.join(words)

reviews['Clean_Review'] = reviews['Reviews'].apply(clean_review)
reviews = reviews[reviews['Clean_Review'].str.len() > 0]
reviews = reviews.drop_duplicates(subset=['Clean_Review'])
reviews[['Clean_Review', 'Sentiment']].head()

```

```

[nltk_data] Downloading package stopwords to
[nltk_data]      /Users/almadani/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!

```

	Clean_Review	Sentiment
0	feel lucky found used phone us not used hard p...	positive
1	nice phone nice grade pantach revue clean set ...	positive
2	pleased	positive
3	works good goes slow sometimes good phone love	positive
4	great phone replace lost phone thing volume bu...	positive

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    reviews['Clean_Review'], reviews['Sentiment'],
    test_size=0.2, random_state=42, stratify=reviews['Sentiment'])

print('Training reviews:', len(X_train))
print('Test reviews:', len(X_test))

```

Training reviews: 122185
Test reviews: 30547

```

from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(max_features=10000)
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

```

```
from sklearn.naive_bayes import MultinomialNB
```

```
nb_classifier = MultinomialNB()
nb_classifier.fit(X_train_tfidf, y_train)
```

```
MultinomialNB()
```

```
from sklearn.metrics import accuracy_score, classification_report
```

```
nb_predictions = nb_classifier.predict(X_test_tfidf)
print('Accuracy:', accuracy_score(y_test, nb_predictions))
print(classification_report(y_test, nb_predictions, zero_division=0))
```

Accuracy: 0.812092840540806

	precision	recall	f1-score	support
negative	0.80	0.74	0.77	8667
neutral	0.28	0.01	0.02	2801
positive	0.82	0.96	0.88	19079
accuracy			0.81	30547
macro avg	0.63	0.57	0.56	30547
weighted avg	0.76	0.81	0.77	30547

```
from sklearn.metrics import confusion_matrix
```

```
labels = ['negative', 'neutral', 'positive']
cm = confusion_matrix(y_test, nb_predictions, labels=labels)
pd.DataFrame(cm,
              index=pd.Index(labels, name='Actual'),
              columns=pd.Index(labels, name='Predicted'))
```

Predicted negative neutral positive

Actual	negative	neutral	positive
negative	6420	24	2223
neutral	875	22	1904
positive	682	32	18365